PaySim Fraud & Risk SQL Practice Set — 60 Questions

A working question set on the PaySim mobile-money dataset, structured the way a fraud ops team might ramp up a new risk analyst: start with basic filtering, build up to the query patterns actually used in transaction monitoring and case review. Each question gets a one-line hint or expected output — try the query first, then check.

Written for **PostgreSQL**. A handful of advanced questions lean on Postgres-specific features (`FILTER`, `PERCENTILE_CONT`, numeric `RANGE` frames, recursive CTEs).

Tags: 🚩 = tied directly to a PaySim fraud scenario (cash-out fraud, velocity, structuring, repeated failures) · 🐛 = debugging task — the SQL shown is broken, fix it.

Schema

`transactions` — the PaySim table

Column	Type	Notes
`step`	int	1 step = 1 simulated hour (744 steps = 30 days)
`type`	text	`CASH_IN`, `CASH_OUT`, `DEBIT`, `PAYMENT`, `TRANSFER`
`amount`	numeric	transaction amount
`nameOrig`	text	origin account (always a customer, starts `C`)
`oldbalanceOrg`	numeric	origin balance before the transaction
`newbalanceOrig`	numeric	origin balance after
`nameDest`	text	destination account (`C` = customer, `M` = merchant)
`oldbalanceDest`	numeric	destination balance before
`newbalanceDest`	numeric	destination balance after
`isFraud`	int	1 = confirmed fraud (PaySim's simulated pattern: account takeover → `TRANSFER` → `CASH_OUT`)
`isFlaggedFraud`	int	1 = the dataset's built-in rule fired (attempted transfer over 200,000 in one shot)

`flagged_accounts` — not part of the original PaySim CSV; a lightweight companion table for the JOIN questions, standing in for a watchlist your team already maintains.

Column	Type	Notes
`account_id`	text	matches `nameOrig` / `nameDest`
`reason`	text	why it's on the watchlist
`flagged_date`	date	when it was added

🟢 Beginner — SELECT, WHERE, GROUP BY, JOIN (Q1–Q20)

Q1. 🚩 Pull every `TRANSFER` over 200,000 — the same threshold the dataset's own flagging rule uses. *Hint:* `WHERE type = 'TRANSFER' AND amount > 200000`.

Q2. Count how many transactions exist for each `type`. *Hint:* `GROUP BY type` — `CASH_OUT` and `PAYMENT` dominate by volume.

Q3. List every transaction where `isFraud = 1`, showing `step`, `type`, `amount`, `nameOrig`, `nameDest`. *Hint:* plain filter — expect almost every row to be `TRANSFER` or `CASH_OUT`.

Q4. Return the 10 largest transactions by amount. *Hint:* `ORDER BY amount DESC LIMIT 10`.

Q5. Total amount moved through each transaction type. *Hint:* `SUM(amount) ... GROUP BY type`.

Q6. 🚩 Find transactions where the origin account's balance hit exactly zero afterward, but wasn't zero before. *Hint:* `WHERE newbalanceOrig = 0 AND oldbalanceOrg > 0` — a classic "account emptied" signature.

Q7. Count how many transactions the built-in rule auto-flagged (`isFlaggedFraud = 1`). *Hint:* expect a tiny number — a flat >200,000 rule catches almost nothing.

Q8. List the distinct transaction types in the dataset. *Hint:* `SELECT DISTINCT type`.

Q9. Find transactions sent to a merchant (`nameDest` starts with `M`) over 50,000. *Hint:* `WHERE nameDest LIKE 'M%' AND amount > 50000`.

Q10. Average transaction amount per type, rounded to 2 decimals. *Hint:* `ROUND(AVG(amount), 2) ... GROUP BY type`.

Q11. How many unique originating accounts are in the dataset? *Hint:* `COUNT(DISTINCT nameOrig)`.

Q12. Pull all transactions from the first simulated day (steps 1–24). *Hint:* `WHERE step BETWEEN 1 AND 24`.

Q13. 🚩 Find fraudulent `CASH_OUT` transactions, largest first. *Hint:* `WHERE type = 'CASH_OUT' AND isFraud = 1 ORDER BY amount DESC` — cash-out is the "get the money out" half of nearly every PaySim fraud case.

Q14. Join to `flagged_accounts` to list transactions where the *origin* is on the watchlist. *Hint:* `JOIN flagged_accounts ON transactions.nameOrig = flagged_accounts.account_id`.

Q15. Count fraudulent vs. non-fraudulent transactions. *Hint:* `GROUP BY isFraud` — expect severe class imbalance.

Q16. 🚩 Find transactions where the amount is bigger than the origin's balance before the transaction — i.e., it shouldn't have gone through. *Hint:* `WHERE amount > oldbalanceOrg`.

Q17. Top 5 accounts by number of transactions sent. *Hint:* `GROUP BY nameOrig ORDER BY COUNT(*) DESC LIMIT 5`.

Q18. Join to `flagged_accounts` to find transactions where the *destination* is on the watchlist, and show the flag reason. *Hint:* `JOIN ... ON transactions.nameDest = flagged_accounts.account_id`.

Q19. Find `PAYMENT` transactions where the destination's balance never moves (`oldbalanceDest = newbalanceDest = 0`) despite a nonzero amount. *Hint:* normal in PaySim — merchant-side balances usually aren't tracked, so this alone isn't a fraud signal.

Q20. 🦕 **Debugging.** Fix this — meant to find `TRANSFER`s over 200,000 but won't run:

```sql
SELECT * FROM transactions
WHERE type = TRANSFER AND amount > 200,000;
```

Hint: `TRANSFER` needs quotes as a string literal, and a numeric literal can't contain a comma (`200000`, not `200,000`).

🟡 Intermediate — Aggregation, Subqueries, Window Functions, Anomaly Detection (Q21–Q40)

Q21. For each account, flag transactions where the amount is more than 3× that account's own average transaction amount. *Hint:* a per-account `AVG(amount)` (subquery or join), then filter `amount > 3 * avg_amount`.

Q22. Running total of `CASH_OUT` amount per account, ordered by `step`. *Hint:* `SUM(amount) OVER (PARTITION BY nameOrig ORDER BY step ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)` — specify `ROWS` explicitly so same-step ties don't get double-counted.

Q23. 🚩 Find accounts making more than 5 transactions within the same `step` — possible scripted/bot behavior. *Hint:* `GROUP BY nameOrig, step HAVING COUNT(*) > 5`.

Q24. 🚩 For each transaction, find how many steps passed since that account's previous transaction; flag gaps under 2 steps as high-velocity. *Hint:* `step - LAG(step) OVER (PARTITION BY nameOrig ORDER BY step)`.

Q25. Find the top 1% of transactions by amount, without hardcoding a dollar threshold. *Hint:* `PERCENT_RANK() OVER (ORDER BY amount) > 0.99`.

Q26. Find transactions where the amount exceeds that account's own average — using a correlated subquery this time, not a window function. *Hint:* `WHERE amount > (SELECT AVG(amount) FROM transactions t2 WHERE t2.nameOrig = t1.nameOrig)`.

Q27. 🚩 Detect structuring: accounts sending 3+ transactions each between 180,000–199,999 (just under the flagging threshold) within a short window. *Hint:* `WHERE amount BETWEEN 180000 AND 199999 GROUP BY nameOrig, step / 24 HAVING COUNT(*) >= 3` — classic smurfing to dodge the >200,000 rule.

Q28. Fraud rate (share of transactions that are fraudulent) by transaction type. *Hint:* `AVG(isFraud::numeric) GROUP BY type` — should be ~0 for everything except `TRANSFER`/`CASH_OUT`.

Q29. Find transactions where the recorded balances don't reconcile: `oldbalanceOrg - amount <> newbalanceOrig`. *Hint:* this shows up disproportionately on fraudulent and merchant-side transactions in PaySim — it's a known quirk of how the simulation writes balances, not a guaranteed fraud tell on its own.

Q30. For each account, return only its single largest transaction. *Hint:* `RANK() OVER (PARTITION BY nameOrig ORDER BY amount DESC)` in a CTE, then filter to `rank = 1`.

Q31. Compute the average gap (in steps) between an account's consecutive transactions — a per-account "normal velocity" baseline. *Hint:* `LAG(step) OVER (PARTITION BY nameOrig ORDER BY step)`, then `AVG(step - prev_step) GROUP BY nameOrig`.

Q32. 🚩 Find `TRANSFER`s immediately followed by a `CASH_OUT` from the same account, one step later — the canonical PaySim fraud chain. *Hint:* self-join or `LEAD()`: match `transfer.nameDest = cashout.nameOrig` at `step + 1`. This is essentially how PaySim's own fraud label was generated.

Q33. Compute a z-score for each transaction's amount relative to its own type, and return anything with `|z| > 3`. *Hint:* `(amount - AVG(amount) OVER (PARTITION BY type)) / STDDEV(amount) OVER (PARTITION BY type)`.

Q34. 🚩 Find accounts with 3 or more transactions where `amount > oldbalanceOrg` — repeated attempts that should have failed on insufficient funds. *Hint:* `WHERE amount > oldbalanceOrg GROUP BY nameOrig HAVING COUNT(*) >= 3`.

Q35. Using a CTE, isolate `TRANSFER`s over 100,000, then find which of those destination accounts went on to `CASH_OUT`. *Hint:* `WITH big_transfers AS (...) SELECT ... FROM transactions JOIN big_transfers ON transactions.nameOrig = big_transfers.nameDest WHERE transactions.type = 'CASH_OUT'`.

Q36. For each `step`, what share of transactions are `CASH_OUT` vs. everything else? *Hint:* `AVG(CASE WHEN type = 'CASH_OUT' THEN 1.0 ELSE 0 END) GROUP BY step`.

Q37. Median transaction amount per type. *Hint:* `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) GROUP BY type`.

Q38. 🐛 **Debugging.** Fix this — meant to show each account alongside its average transaction amount:

```sql
SELECT nameOrig, amount, AVG(amount)
FROM transactions
GROUP BY nameOrig;
```

Hint: `amount` is neither grouped nor aggregated, so Postgres rejects it (`column "transactions.amount" must appear in the GROUP BY clause or be used in an aggregate function`). Drop it from the `SELECT`, or use a window function instead of `GROUP BY`.

Q39. 🐛 **Debugging.** Fix this — meant to flag accounts with more than 10 transactions:

```sql
SELECT nameOrig, COUNT(*) AS txn_count
FROM transactions
WHERE COUNT(*) > 10
GROUP BY nameOrig;
```

Hint: aggregate conditions belong in `HAVING`, not `WHERE` (`aggregate functions are not allowed in WHERE`) — `WHERE` runs before grouping happens.

Q40. 🐛 **Debugging.** This is meant to rank each account's transactions by amount, but every row comes back ranked 1:

```sql
SELECT nameOrig, amount,
  RANK() OVER (PARTITION BY nameOrig) AS amt_rank
FROM transactions;
```

Hint: no `ORDER BY` inside `OVER()` — without one, every row in the partition is a single tied peer group, so all get rank 1.

🔴 Advanced — Fraud Chains, Multi-Signal Risk, Scale (Q41–Q60)

Q41. Using a recursive CTE, trace transaction chains up to 3 hops (A→B→C): an account receives, then forwards. *Hint:* base case = every transaction at `depth = 1`; recursive case joins `chain.nameDest = transactions.nameOrig`, `depth + 1`, stop at `depth < 3`.

Q42. 🚩 Identify possible mule accounts: high fan-in (many distinct senders) followed by a fast, near-total cash-out. *Hint:* `GROUP BY nameDest` with `COUNT(DISTINCT nameOrig)`, joined to a check that a `CASH_OUT` from that same account follows within a few steps for a comparable amount.

Q43. 🚩 Flag accounts with 3+ over-limit attempts (`amount > oldbalanceOrg`) inside any rolling 24-step window. *Hint:* `COUNT(*) OVER (PARTITION BY nameOrig ORDER BY step RANGE BETWEEN 24 PRECEDING AND CURRENT ROW)` — Postgres supports numeric `RANGE` offsets directly on an integer `step` column.

Q44. 🚩 Identify high-risk merchants: destinations that receive disproportionately often from accounts with a fraud history, even though the merchant itself is never `isFraud = 1`. *Hint:* join merchant-received transactions back to `SUM(isFraud) OVER (PARTITION BY nameOrig)` for the sending account.

Q45. For each account, compute a running average and running max of transaction amount up to and including the current one (ordered by `step`) — "how does this compare to what I've seen from this account so far," which is the shape real-time scoring actually needs. *Hint:* `AVG(amount) OVER (PARTITION BY nameOrig ORDER BY step ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`, same frame for `MAX`. (Note: ranking functions like `PERCENT_RANK` ignore frame clauses entirely — ordinary aggregates like `AVG`/`MAX`/`COUNT` are what respect them.)

Q46. 🚩 Detect a structuring ring: 3+ distinct origin accounts all paying the same destination, each just under 200,000, within a tight step window. *Hint:* filter to `amount BETWEEN 150000 AND 199999`, then `GROUP BY nameDest, step / 24 HAVING COUNT(DISTINCT nameOrig) >= 3`.

Q47. Write a single-pass query — no correlated subquery — that returns every account's transaction count, total amount, and fraud rate. It needs to hold up on 100M+ rows. *Hint:* one `GROUP BY nameOrig` with conditional aggregation (`SUM(amount)`, `AVG(isFraud::numeric)`); make sure `nameOrig` is indexed.

Q48. Find the longest transaction chain in the dataset using a recursive CTE with a cycle guard. *Hint:* carry a visited-accounts array (or cap `depth`) in the recursive term so a cycle (A→B→A) can't recurse forever.

Q49. Build a composite risk score per transaction from amount z-score, account velocity, and destination type (merchant vs. customer), then return the top 100. *Hint:* compute each

signal as its own window-function column in a CTE, combine into a weighted sum, `ORDER BY score DESC LIMIT 100`.

Q50. 🚩 Find accounts that go dormant for 500+ steps and then suddenly transact heavily — a classic account-takeover signature. *Hint:* `step - LAG(step) OVER (PARTITION BY nameOrig ORDER BY step) > 500`, then compare transaction frequency right after that gap.

Q51. Find account pairs that repeatedly transact back and forth (A→B, then B→A) within short step windows — possible layering. *Hint:* self-join `transactions` to itself on swapped `nameOrig`/`nameDest` with a small step-difference bound.

Q52. 🐛 **Debugging.** This is meant to be a fast lookup on `CASH_OUT` transactions but runs a full table scan even with an index on `type`:

```sql
SELECT * FROM transactions WHERE UPPER(type) = 'CASH_OUT';
```

Hint: wrapping the column in `UPPER()` makes the predicate non-sargable — the plain index on `type` can't be used. Compare the raw value (`type = 'CASH_OUT'`) or build an expression index on `UPPER(type)`.

Q53. Sketch a query (and the reasoning) for scoring only *new* transactions in a nightly batch job, instead of rescanning the full table every run. *Hint:* `WHERE step > (SELECT MAX(step) FROM last_batch_log)`; at real scale, range-partition `transactions` by `step` so old partitions are never touched.

Q54. 🚩 Compare `isFlaggedFraud` against `isFraud`: how many transactions did the flat >200,000 rule miss, and how many did it flag that weren't actually fraud? *Hint:* conditional counts — `isFlaggedFraud=1 AND isFraud=0` (false positives) vs. `isFraud=1 AND isFlaggedFraud=0` (missed fraud). This is precision/recall for one hardcoded rule, and it's a rough one.

Q55. 🚩 For each account with at least one fraudulent transaction, find the gap (in steps) between its very first transaction and its first fraudulent one. *Hint:* `MIN(step) FILTER (WHERE isFraud = 1) - MIN(step) ... GROUP BY nameOrig`.

Q56. 🐛 **Debugging.** Meant to find merchants receiving from many distinct fraud-linked accounts — errors out, and the approach is wrong even fixed naively:

```sql
```

```sql
SELECT nameDest, COUNT(*)
FROM transactions t1, transactions t2
WHERE t1.nameOrig = t2.nameOrig
AND t1.isFraud = 1
GROUP BY nameDest;
```

Hint: `nameDest` is ambiguous — it exists in both `t1` and `t2` (`column reference "namedest" is ambiguous`). But the real problem is the self-join itself; it doesn't relate to the question at all. Drop it: `SELECT nameDest, COUNT(DISTINCT nameOrig) FROM transactions WHERE isFraud = 1 GROUP BY nameDest`.

Q57. 🐛 Debugging. This recursive CTE for tracing transaction chains never finishes:

```sql
WITH RECURSIVE chain AS (
  SELECT nameOrig, nameDest, amount, 1 AS depth
  FROM transactions
  UNION ALL
  SELECT c.nameOrig, t.nameDest, t.amount, c.depth + 1
  FROM chain c
  JOIN transactions t ON c.nameDest = t.nameOrig
)
SELECT * FROM chain;
```

Hint: no depth cap and no cycle check — any A→B→A loop in the data recurses indefinitely. Add `WHERE c.depth < 5` and/or track visited accounts.

Q58. 🐛 Debugging. Meant to return the top 3 accounts with more than 3 over-limit attempts — comes back completely empty even though qualifying accounts exist:

```sql
SELECT nameOrig, COUNT(*)
FROM transactions
WHERE amount > oldbalanceOrg
GROUP BY nameOrig
HAVING COUNT(*) > 3
ORDER BY COUNT(*) DESC
LIMIT 3
OFFSET 5;
```

Hint: `OFFSET 5` skips the first 5 ranked rows before `LIMIT 3` even applies — if fewer than 6 accounts qualify, you get nothing back. That `OFFSET` looks like leftover pagination code;

drop it.

Q59. 🦎 Debugging. This 3-transaction rolling average, meant for a real-time scoring pipeline, leaks future data:

```sql
SELECT nameOrig, step, amount,
  AVG(amount) OVER (
    PARTITION BY nameOrig ORDER BY step
    ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING
  ) AS rolling_avg
FROM transactions;
```

Hint: `AND 3 FOLLOWING` pulls in transactions that haven't happened yet — fine for offline analysis, but a live pipeline can't see the future. Use `ROWS BETWEEN 3 PRECEDING AND CURRENT ROW`.

Q60. Write a single query that assembles an investigator-ready case file for one account: total transaction count and volume, its largest single transaction, its fraud-flag history, and any `TRANSFER → CASH_OUT` chain it took part in — everything a human reviewer would want on one screen before deciding whether to escalate. *Hint:* a few CTEs (one for aggregate stats, one for the chain check from Q32/Q41) combined in a final `SELECT`. This is closer to what actually gets built for case-management tooling than any single flat query — manual review still needs a real query behind it.

Using this set

Work top to bottom the first time through — later questions lean on earlier patterns (Q45's running window builds on Q22's; Q41's chain CTE generalizes Q32's fraud pattern). For the advanced debugging questions, run `EXPLAIN ANALYZE` on your fix rather than just checking correctness — a few of these are broken in ways that are slow, not wrong.